

# Agent Benchmarking Handbook

Version 0.3 release candidate - 2026-07-13

## Purpose

This handbook defines how to determine whether Agentic Work Best Practices improves real engineering outcomes. Routing tests, attractive reports and green public tests are not sufficient. A valid result requires independent final-state grading, fixed controls, isolated hidden material, retained failures and the actual model, effort, cost and environment.

| Arm                     | Context                                    | Enforcement                           |
|-------------------------|--------------------------------------------|---------------------------------------|
| vanilla                 | Provider defaults and fixture instructions | none                                  |
| core                    | Fixture plus minimal Stable Core           | none                                  |
| core-router             | Core plus deterministic Context Pack       | none                                  |
| core-router-enforcement | Core, Context Pack and tested gates        | protected inputs and completion gates |

Use the same provider, resolved model, effort, prompt, clean fixture, permissions and budget for every arm. Compare model tiers only after the harness experiment so model quality and harness quality are not confounded.

## Current readiness

Twelve private fixtures are executable. For every fixture, the flawed starter passes its public tests and is rejected by the hidden grader; the reference solution passes public and hidden checks. The outcome-harness proof is 12/12.

A dedicated WSL2 distribution named `AgenticBench` is provisioned separately from the user's normal Ubuntu environment. It has a single unprivileged user, private home permissions, no `sudo`, no mounted Windows drive and no Windows executable interop. The provider process receives only one public run workspace under `/srv/agenticbench/workspace`; hidden graders and this source repository remain on the Windows host.

Codex CLI and Claude Code are installed user-locally inside the dedicated distribution. Their exact versions are discovered and written only to ignored local status. Authentication is a one-time OAuth step and remains a hard preflight gate; no provider-backed run starts until both boolean login checks pass.

The first subscription-backed smoke was retained as a **non-publishable infrastructure shakeout**, not scored as a model or harness comparison. Five provider invocations occurred: two sequential attempts completed, two campaign-linked attempts overlapped, and a fifth orphan attempt was created by a campaign-state race. The affected attempts are explicitly marked `invalid_infrastructure`; the entire campaign is closed as `closed-diagnostic-invalid`. This failure produced regression controls for exclusive campaign/provider locks, orphan detection, closed-campaign refusal and protected-file enforcement.

The subsequent repaired smoke completed exactly four sequential calls with no replacement: both Codex and Claude, each in `vanilla` and `core-router-enforcement`. All four passed public tests, hidden grading at 100/100 and applicable enforcement; there were zero overlaps and zero invalid attempts. The result proves harness viability but is **INCONCLUSIVE** about comparative lift because every cell has only one trial and all reached the score ceiling. Codex `provider-default` did not report an exact resolved model, so directional remains unapproved.

## Release gates

Before spending money:

```
powershell -ExecutionPolicy Bypass -File tools/release-gate.ps1 -Mode infrastructure
```

This validates JSON and PowerShell, both routers, promotion/rollback, loops, handoffs, fixture discrimination, run lifecycle, benchmark concurrency/model-snapshot safety, security hooks, Windows/OneDrive and doctor. The post-incident infrastructure gate passed 13/13 checks.

The full gate uses `tools/release-gate.ps1 -Mode full`. It remains red until all objective requirements have evidence and twelve private fixtures are executable. Do not weaken it to make the repo appear complete.

## Benchmark ladder

### Private failure-oriented fixtures

Use private fixtures for the failures this repo targets: sample hardcoding, regex brittleness, undefined symbols, downstream breakage, requirement omission, weakened tests, prompt injection, migration safety, cold resume and false completion. This is the strongest causal test because only the harness arm changes.

### Terminal-Bench 2.0

Use [Terminal-Bench 2.0 and Harbor](#) for hard terminal tasks in containers. Harbor's common container interface fits host-only grading. Start with a stratified subset and record task IDs, versions, image digests and adapter commit.

### OpenHands Index

Use the [OpenHands Index](#) to triangulate issue resolution, greenfield, frontend, testing and information-gathering performance versus cost. Its SDK harness differs from this repo, so it is external context, not causal evidence that this Core helped.

### SWE-bench family

Do not use SWE-bench Verified as the primary frontier metric. OpenAI reported in 2026 that it is increasingly contaminated and that many audited failures involved flawed tests, recommending SWE-bench Pro instead. See [OpenAI's assessment](#).

Verified may remain a disclosed regression lane. The [official SWE-bench guide](#) documents variants and instance fields. Never present a contaminated public score as the primary real-world proof.

### Owned-repository retrospectives

This is the highest-value product lane. Select historical Vextrum and Hermes tasks with a recoverable pre-change commit, original request, known accepted behavior, independent invariants and no production side effects. Grade behavior and compatibility, not byte-identical reproduction of the historical patch. Keep gold patches and new tests outside the agent environment.

### Prospective shadow mode

After retrospective success, run new work in isolated branches or worktrees while outward actions remain human-gated. Measure review time, corrections, escaped defects and whether work was accepted, heavily revised or discarded.

Capability is not productivity. METR identifies task selection, participant selection, multi-agent time accounting and final-quality differences as material measurement problems. See [METR's 2026 design update](#).

## Secure hidden grading

A hidden result is publishable only when:

1. The agent receives only the public workspace.
2. The benchmark repo, grader, reference solution and other arms are unreadable.
3. Network access cannot retrieve hidden material.
4. The host grades only after the agent exits.
5. Workspace and grader hashes are recorded.
6. Container, VM or dedicated-WSL isolation evidence is saved.

Directory separation is `logical-only`: useful for harness development, never sufficient for a published hidden score. AgenticBench qualifies only while live doctor evidence confirms that `/mnt/c` is not mounted and the host repo is unavailable. The run manifest records the isolation type automatically.

Provider OAuth is an unavoidable local capability of subscription CLIs. Therefore it is confined to the empty benchmark distribution rather than copied from a personal home. The agent can affect only its current public workspace; returned files are rejected if they contain symlinks, special filesystem objects, credential filenames, credential-shaped content, excessive file counts or excessive bytes. Authentication files, raw status output, account identifiers and session data

never enter Git artifacts.

## Subscription setup

Codex can use an eligible ChatGPT subscription and Claude Code can use a Claude Pro or Max subscription. This avoids API keys, but it does not make runs unlimited: usage consumes the same subscription allowances used by the corresponding applications. See [OpenAI's Codex plan guidance](#) and [Anthropic's Pro/Max Claude Code guidance](#).

From a fresh clone, run the one-time setup:

```
powershell -ExecutionPolicy Bypass -File Setup/benchmark.ps1 `
-Create -LoginCodex -LoginClaude -RefreshProviderCatalog
```

The two logins require human OAuth once. Normal restarts do not require setup or login. Immediately before spending subscription capacity, run:

```
powershell -ExecutionPolicy Bypass -File tools/preflight.ps1 -Mode benchmark
```

The preflight fails closed on expired provider snapshots, either missing login, provider CLI failure or isolation drift.

## The staged 40-run pilot

2 fixtures x 2 providers x 2 arms x 5 trials = 40 runs

The causal comparison is intentionally narrow: `vanilla` versus `core-router-enforcement`. The campaign adds complexity only after the primary effect is understood.

| Stage       | New runs | Cumulative | Purpose                                           |
|-------------|----------|------------|---------------------------------------------------|
| smoke       | 4        | 4          | one fixture, both providers, both arms, one trial |
| directional | 4        | 8          | add the second fixture at one trial               |
| confidence  | 16       | 24         | add trials 2-3 to every cell                      |
| pilot       | 16       | 40         | add trials 4-5 to every cell                      |

Create the randomized campaign without starting a provider:

```
powershell -ExecutionPolicy Bypass -File Evals/tools/new-benchmark-campaign.ps1
```

Approve and run only one stage at a time:

```
powershell -ExecutionPolicy Bypass -File Evals/tools/approve-benchmark-stage.ps1 `
-Campaign <campaign-id> -Stage smoke -ApprovedBy <human>

powershell -ExecutionPolicy Bypass -File Evals/tools/run-benchmark-stage.ps1 `
-Campaign <campaign-id> -Stage smoke -MaxRunsThisInvocation 4
```

The default loop manifest allows at most four runs per invocation, 15 wall-clock minutes and 12 Claude turns per run, two consecutive failures, two no-progress outcomes and 40 scheduled runs total. Creating `Evals/local/STOP` is the kill switch. Authentication loss, provider drift, isolation failure or any ceiling stops before the next run.

Only one campaign runner and one provider adapter may own the fixed WSL workspace. Both use fail-closed exclusive locks. A second runner must stop before preparing or invoking a provider. The adapter also refuses to reset the workspace while an orphan `codex`, `claude` or `AgenticBench` runner process exists. Never start a second terminal to “resume” a stage whose first terminal disappeared; inspect and recover or invalidate the active attempt first.

`Evals/local/provider-settings.json` is ignored, contains no credentials and expires after seven days. `provider-default` is acceptable only for smoke discovery. Every post-smoke stage requires explicit model selectors so the comparison cannot silently mix model versions. Provider events are inspected for the actual resolved model; unresolved defaults are recorded honestly as `unresolved-provider-default`, never guessed. Refreshing a catalog never changes a model default outside the ignored local snapshot.

Each successful provider process is followed by deterministic public tests and a host-only hidden grader. Failures, timeouts and refusals are retained; the harness never retries only the losing arm.

## Controls and metrics

Hold constant: exact model snapshot, effort, tool permissions, network policy, task prompt, fixture hash, budgets, retry/stop policy, grader and container image. Randomize order. Retain timeouts, refusals and failures. Never rerun only failed arms unless the same rule applies to every arm.

Primary metrics: hidden final-state pass, unseen-input generalization, security-critical misses, omitted requirements, downstream regressions and false completion claims.

Secondary metrics: public tests, unnecessary changes, protected-file edits, wall time, tokens, cost, tool failures, interventions and human review minutes.

Report counts and distributions, not only averages. Five trials are a minimum stability check, not high statistical power. When intervals are wide or differences small, conclude *inconclusive*.

## Promotion rules

Additional complexity is eligible only when it causes no security-critical regression, improves hidden correctness/generalization or enforces a documented safety invariant, does not worsen omission/false-done rates, survives all required cases, discloses cost/latency/review tradeoffs and has rollback.

Model routing is separate: choose the harness with one fixed model, then compare capability tiers on that arm. A cheaper model is eligible only when its quality floor is preserved. High-risk work retains its capability floor and human gate.

## Fixture standard

Every private fixture needs realistic public state, observable final behavior, incomplete public tests, hidden unseen/negative/adversarial checks, a 100% reference solution, a negative control accepted publicly but rejected privately, stable requirement IDs, deterministic bounded grading, no production side effects and a leakage review.

Reject fixtures that enforce undocumented implementation details, reject valid alternatives, depend on flaky services or reward memorization.

Before acceptance, a second reviewer must inspect the task-to-test alignment and confirm that behaviorally valid alternative implementations can pass. Record that review with the fixture version so later grader changes remain auditable.

## Failure handling

- Pre-agent environment failure: invalidate and rerun all affected conditions under one rule.
- Agent timeout or budget stop: score failure.
- Provider outage: infrastructure failure unless reliability is the target.
- Hidden leakage: invalidate the experiment.
- Existing protected test or task modification/deletion: fail enforcement. New regression tests are allowed; generated `__pycache__` and `.pyc` files are ignored.
- Concurrent or orphan provider execution: preserve raw evidence, mark every affected attempt `invalid_infrastructure`, close a confounded campaign if comparability was lost and require a fresh human-approved campaign.
- Grader defect: freeze, version the fix and rerun every affected arm.
- Model drift mid-experiment: stop; never mix resolved versions.

## Required output

Every experiment produces structured JSON, Markdown and a visually verified PDF with hypotheses, versions/hashes, isolation evidence, budgets, all dispositions, uncertainty, quality/cost/latency/review analysis, invalidations, recommendation, limitations, rollback implications and clickable sources. Structured records are the measurement source of truth; PDF is the human artifact.

## Sources

- [Terminal-Bench 2.0 and Harbor](#)
- [OpenHands Index](#)
- [OpenAI SWE-bench Verified assessment](#)
- [Official SWE-bench dataset guide](#)
- [METR 2026 experiment-design update](#)
- [ContextBench](#)
- [Evaluating AGENTS.md](#)
- [Probe-and-Refine Repository Guidance](#)
- [OpenAI: Using Codex with a ChatGPT plan](#)
- [Anthropic: Using Claude Code with Pro or Max](#)